Analysis of Cache Size and Associativity in RISC-V Using GEM5

Harsh Patel, Het Patel
Institute of Technology, Nirma University
Ahmedabad, India
23BEC134, 23BEC135

***Abstract*—The performance of a processor is strongly influenced by the design of its first-level (L1) data cache, and two of the most fundamental design parameters are cache size and associativity. This report presents an empirical, simulation-based analysis of the effect of L1 data cache (L1D) size and associativity on the performance of a RISC-V processor, using the cycle-level GEM5 architectural simulator. A RISC-V system is modeled with a `RiscvTimingSimpleCPU`, a fixed 16 KB, 2-way L1 instruction cache, a fixed 256 KB, 8-way L2 cache, and a variable L1 data cache, executing a custom C workload compiled for RISC-V. Two controlled experiments are performed: in the first, L1D size is swept over 8, 16, 32, and 64 KB at a fixed 4-way associativity; in the second, L1D associativity is swept over 2, 4, 8, and 16-way at a fixed 16 KB size. Only one parameter is changed at a time so that its individual effect on hit rate, miss rate, and cycles-per-instruction (CPI) can be isolated. Results show that increasing L1D size from 8 KB to 64 KB raises the hit rate from 92.08% to 99.98% and reduces CPI by 15.0%, while increasing associativity from 2-way to 16-way at a fixed 16 KB size raises the hit rate from 95.65% to 99.41% and reduces CPI by 8.76%. These results are consistent with the classical relationship between cache size, capacity misses, associativity, and conflict misses, and are discussed in the context of related GEM5-based cache studies.**

***Index Terms*—RISC-V, GEM5, cache memory, cache size, set-associative cache, cache hit rate, CPI, architectural simulation**

I. Introduction

Modern processors rely heavily on cache memory to bridge the widening speed gap between the CPU and main memory. Because main memory access latency has not scaled at the same rate as processor clock speed, a small and fast cache placed between the CPU and main memory is used to hold frequently accessed data, allowing most memory references to be serviced without incurring a slow main-memory access [1]. The effectiveness of this cache is governed chiefly by two design parameters: its *size*, which determines how much data it can hold before capacity misses occur, and its *associativity*, which determines how many locations within the cache a given main-memory block may occupy and therefore how much conflict-miss behavior the cache exhibits [1].

RISC-V, as an open instruction-set architecture, is increasingly used as a research and teaching vehicle for computer-architecture experimentation, and the GEM5 simulator provides a flexible, cycle-level platform on which such experiments can be conducted without the cost of fabricating hardware [2]. GEM5 exposes cache size, associativity, and other memory-hierarchy parameters directly through its Python configuration interface, and reports detailed statistics – including per-level hits, misses, and CPI – in its `stats.txt` output file after every run. This makes it well suited to controlled, single-parameter sweeps of the kind performed in this work.

This report studies the L1 data cache of a RISC-V system specifically, since data caches tend to be more sensitive to workload-driven capacity and conflict pressure than instruction caches [4]. Two experiments are designed: an L1D *size* sweep at fixed associativity, and an L1D *associativity* sweep at fixed size. In each experiment, exactly one parameter is varied while every other system parameter – CPU model, clock, L1 instruction cache, L2 cache, and workload – is held fixed, so that the measured change in hit rate, miss rate, and CPI can be attributed to the parameter under test.

The remainder of this report is organized as follows. Section II reviews related work on GEM5-based cache characterization. Section III summarizes the cache-memory background relevant to this study. Section IV describes the proposed experimental methodology, including the simulated RISC-V system and workloads. Section V presents and discusses the measured results. Section VI discusses the limitations of this study and directions for future work, and Section VII concludes the report.

II. Related Work

GEM5 has been widely used to characterize processor and cache behavior because it exposes a modular, configurable memory system through a Python front end while still producing detailed, cycle-accurate statistics [2].

Sawal [3] modeled a simple single-core X86 system in GEM5, first without any cache hierarchy and then with a two-level (L1 instruction/data and L2) cache hierarchy, and compared the resulting memory-access behavior across two DRAM types (DDR3 and DDR4) and two clock frequencies. That work established the basic GEM5 workflow used in this report – building a configuration script, running the simulation in System-call Emulation (SE) mode, and extracting statistics from `stats.txt` – but did not vary cache size or associativity independently, nor did it target a RISC-V system.

Akula *et al.* [4] performed a large-scale empirical study (over 500 simulation runs) of L1 instruction-cache (L1I) and L1 data-cache (L1D) size on an X86 system executing the Freqmine benchmark from the PARSEC suite, using GEM5's

Ruby memory model and an out-of-order (O3) CPU in Full-System mode. Sweeping both L1I and L1D from 16 KB to 1024 KB, they found that L1D size had a strong, direct effect on hit rate, miss count, and instructions-per-cycle, while L1I size had no measurable effect for their data-intensive workload, and that below a threshold L1D size the simulated system failed to complete the benchmark. Their study confirms the expected direct relationship between data-cache size and hit rate at a much larger scale than the present work, but does not examine associativity as an independent variable, and targets an X86, PARSEC-benchmarked system rather than a RISC-V system with a controlled synthetic workload.

The present report differs from both of the above in three respects: (i) it targets a RISC-V processor (`RiscvTimingSimpleCPU`) rather than X86, (ii) it studies L1D size and L1D associativity as two *separate*, single-variable experiments rather than a joint sweep, allowing each parameter's effect to be isolated, and (iii) it uses small, purpose-built C workloads – one designed to make cache-size effects observable through a 64 KB pseudo-random access pattern, and one designed to make associativity/conflict effects observable through accesses to blocks separated by exactly 16 KB – rather than a general data-mining benchmark.

III. Background: Cache Size and Associativity

A cache holds a subset of main memory in fixed-size lines. Three classical categories of cache miss are relevant to this study [1]. A *compulsory miss* occurs on the first access to a line that has never been brought into the cache. A *capacity miss* occurs when the cache is too small to hold all of the data required by the program at once, so that a previously cached, still-needed line has to be evicted to make room for another. A *conflict miss* occurs when the cache's placement (mapping) scheme forces a line to be evicted even though the cache as a whole has free space, simply because multiple blocks compete for the same set.

Cache *size* directly determines a cache's susceptibility to capacity misses: a larger cache can hold more of a program's working set, reducing the rate at which useful lines must be evicted purely due to limited space [1]. Cache *associativity* determines how many distinct locations (*ways*) within a set a given main-memory line may occupy. A direct-mapped cache (1-way) allows only one placement location per line and is therefore maximally prone to conflict misses; an N-way set-associative cache allows N candidate locations per set, reducing conflict misses at the cost of additional per-access tag comparison hardware [1]. Because cache size targets capacity behavior and associativity targets conflict behavior, this report studies them as two independent experiments rather than as a single joint parameter.

IV. Proposed Experimental Methodology

A. Simulated RISC-V System

The system under test is configured in GEM5 (version 23.0.0.1) for the RISC-V architecture and simulated in System-call Emulation (SE) mode. Table I summarizes the fixed system configuration used throughout both experiments.

TABLE I
Fixed GEM5/RISC-V System Configuration

Parameter	Value
GEM5 version	23.0.0.1
Architecture	RISC-V
Simulation mode	System-call Emulation (SE)
CPU model	`RiscvTimingSimpleCPU`
Clock frequency	1 GHz
Memory mode	Timing
Main memory	DDR3_1600_8x8, 512 MB range
L1 instruction cache (L1I)	16 KB, 2-way (fixed)
L2 cache	256 KB, 8-way (fixed)
L1 data cache (L1D)	*Variable – see Sections IV-B, IV-C*

The GEM5 executable used is `build/RISCV/gem5.opt`, invoked with a Python configuration script that instantiates the CPU, the `SystemXBar` interconnect, the memory controller, and the cache objects, and specifies the compiled RISC-V workload binary. After each run, GEM5 writes an `m5out/` directory containing `stats.txt` (instructions, CPU cycles, CPI, and per-level cache accesses/hits/misses) and `config.ini`, which is inspected after every run to confirm that the intended L1D size and associativity were actually applied to the simulated system.

B. Experiment 1: L1D Cache Size Sweep

In the first experiment, L1D associativity is held fixed at 4-way while L1D size is swept over four values: 8 KB, 16 KB, 32 KB, and 64 KB. All other parameters in Table I are held fixed. The workload for this experiment operates on a 64 KB data array using a repeated, pseudo-random access pattern over `volatile` data, chosen specifically so that a data set larger than the smallest cache configurations under test would produce an observable difference in capacity-miss behavior as L1D size is increased.

C. Experiment 2: L1D Associativity Sweep

In the second experiment, L1D size is held fixed at 16 KB while associativity is swept over four values: 2-way, 4-way, 8-way, and 16-way. All other parameters in Table I are held fixed. The workload for this experiment is conflict-oriented: it accesses multiple memory blocks separated by exactly 16 KB (matching the fixed L1D size) in access groups of 2, 4, 8, and 16 blocks, so that a low-associativity cache is forced to evict blocks that a higher-associativity cache of the same total size could retain simultaneously.

D. Measured Metrics

For both experiments, the following metrics are extracted from `stats.txt` for each configuration:

- **Instructions** – total instructions executed;
- **CPU Cycles** – total simulated clock cycles;
- **L1D Hits / L1D Misses** – demand accesses satisfied / not satisfied by L1D;

- **CPI** = CPU Cycles/Instructions;
- **Hit Rate** = Hits/(Hits + Misses) × 100;
- **Miss Rate** = Misses/(Hits + Misses) × 100.

Only one cache parameter is changed at a time across each experiment's runs, so that the measured change in these metrics can be attributed to that parameter alone.

V. Results and Discussion

A. Effect of L1D Cache Size

Table II reports the measured statistics for the L1D size sweep at a fixed 4-way associativity, and Fig. 1(a) and Fig. 2(a) plot the resulting hit/miss rate and CPI trends.

TABLE II
MEASURED RESULTS – L1D SIZE SWEEP (4-WAY, FIXED)

L1D Size	**8 KB**	**16 KB**	**32 KB**	**64 KB**
Instructions		118,196,803 (constant)		
CPU Cycles	456,213,344	446,613,512	426,606,608	387,650,120
L1D Hits	33,279,880	33,679,873	34,513,494	36,136,682
L1D Misses	2,864,233	2,464,240	1,630,619	7,431
CPI	3.8598	3.7786	3.6093	3.2797
Hit Rate (%)	92.0755	93.1822	95.4886	99.9794
Miss Rate (%)	7.9245	6.8178	4.5114	0.0206

As L1D size increases from 8 KB to 64 KB, hit rate rises monotonically from 92.08% to 99.98%, while miss rate falls from 7.92% to 0.02% – a relative reduction of over 99%. The sharpest drop in miss rate occurs between 32 KB and 64 KB, where the cache becomes large enough to hold the entire 64 KB working set of the size-sweep workload, eliminating almost all capacity misses at once. CPI decreases correspondingly, from 3.8598 at 8 KB to 3.2797 at 64 KB, a 15.0% reduction, since fewer L1D misses mean fewer stall cycles spent waiting on the L2 cache and main memory. These trends are consistent with the expected relationship between cache size and capacity misses described in Section III, and with the direct size-to-hit-rate relationship reported for X86/PARSEC workloads by Akula *et al.* [4].

B. Effect of L1D Associativity

Table III reports the measured statistics for the L1D associativity sweep at a fixed 16 KB size, and Fig. 1(b) and Fig. 2(b) plot the resulting trends.

TABLE III
MEASURED RESULTS – L1D ASSOCIATIVITY SWEEP (16 KB, FIXED)

Associativity	**2-way**	**4-way**	**8-way**	**16-way**
Instructions		1,860,100 (constant)		
CPU Cycles	7,667,636	7,571,636	7,379,676	6,996,020
L1D Hits	711,655	715,655	723,655	739,639
L1D Misses	32,379	28,379	20,379	4,395
CPI	4.1222	4.0706	3.9674	3.7611
Hit Rate (%)	95.6482	96.1858	97.2610	99.4093
Miss Rate (%)	4.3518	3.8142	2.7390	0.5907

As associativity increases from 2-way to 16-way at a fixed 16 KB size, hit rate rises monotonically from 95.65% to 99.41%, and miss rate falls from 4.35% to 0.59% – a relative reduction of about 86%. Because the associativity-sweep workload deliberately accesses blocks separated by exactly 16 KB, a low-associativity cache is repeatedly forced to evict a block it will need again shortly afterward, which is precisely the conflict-miss behavior described in Section III; increasing associativity gives each set more candidate locations and directly reduces this effect. CPI decreases from 4.1222 at 2-way to 3.7611 at 16-way, an 8.76% reduction – smaller than the 15.0% CPI reduction observed for the size sweep, indicating that, for the workloads used here, capacity misses have a somewhat larger impact on overall CPI than conflict misses do, though both parameters produce a clear and monotonic improvement in cache performance.

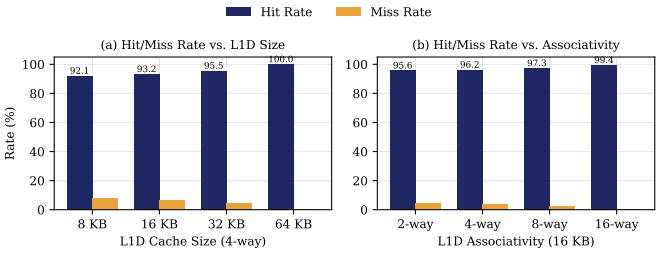


Fig. 1. Measured L1D hit rate and miss rate: (a) vs. cache size at fixed 4-way associativity; (b) vs. associativity at fixed 16 KB size.

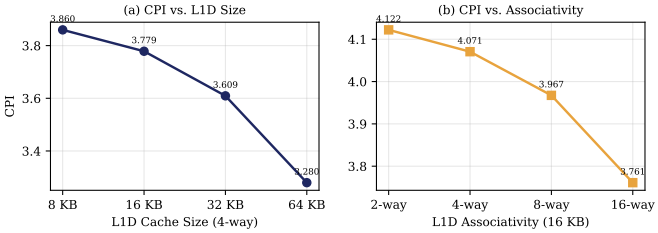


Fig. 2. Measured CPI: (a) vs. cache size at fixed 4-way associativity; (b) vs. associativity at fixed 16 KB size.

C. Comparison with Related GEM5 Cache Studies

The direct, monotonic relationship between L1D size and hit rate observed here matches the trend reported by Akula *et al.* [4] for X86/PARSEC workloads, despite the differences in ISA, CPU model, and workload, suggesting that this relationship is a general property of set-associative caches rather than an artifact of a specific benchmark. Unlike that study, however, the present work additionally isolates associativity as an independent variable, which shows that – for a workload specifically designed to induce conflict misses – associativity alone can move hit rate from 95.65% to 99.41% without any change in total cache capacity.

VI. Limitations and Future Work

This study fixes the L1 instruction cache and L2 cache configuration throughout both experiments; the interaction

between L1D configuration and L2 behavior (e.g., L2 hit rate as L1D size or associativity changes) is not analyzed and is left for future work. The two workloads used are small, purpose-built C programs designed to make size- and associativity-related effects clearly observable, rather than standardized benchmarks; validating these trends against a standardized suite such as PARSEC, in the manner of Akula *et al.* [4], would strengthen the generality of the conclusions. The associativity sweep is limited to a single fixed L1D size (16 KB); repeating it at other sizes would clarify whether the relative impact of associativity changes with cache capacity. Finally, this report considers L1D size and associativity only in isolation; a joint sweep, or an analysis of other cache design parameters such as line size and replacement policy, is identified as a direction for future work.

VII. Conclusion

This report presented an empirical, GEM5-based analysis of the effect of L1 data cache size and associativity on the performance of a RISC-V processor. Two controlled experiments were performed, changing one cache parameter at a time. Increasing L1D size from 8 KB to 64 KB at a fixed 4-way associativity raised hit rate from 92.08% to 99.98% and reduced CPI by 15.0%, while increasing L1D associativity from 2-way to 16-way at a fixed 16 KB size raised hit rate from 95.65% to 99.41% and reduced CPI by 8.76%. These results are consistent with the classical view that cache size primarily governs capacity-miss behavior while associativity primarily governs conflict-miss behavior, and GEM5 was shown to be an effective, controllable platform for isolating and quantifying these two effects on a RISC-V system.

References

[1] B. Parhami, *Computer Architecture: From Microprocessors to Supercomputers*, Oxford University Press, 2005

[2] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti, R. Sen, K. Sewell, M. Shoaib, N. Vaish, M. D. Hill, and D. A. Wood, "The gem5 simulator," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 2, pp. 1–7, May 2011.

[3] S. N. Sawal, "Performance Analysis of the Designed CPU Using GEM5 Simulator," in *Proc. 8th Int. Conf. Recent Trends in Engineering, Science and Management (ICRTESM-17)*, Phagwara, Punjab, India, May 2017, pp. 398–402.

[4] R. Akula, K. Jain, and D. J. Kotecha, "System Performance with Varying L1 Instruction and Data Cache Sizes: An Empirical Analysis," *arXiv:1911.11642v1 [cs.PF]*, Nov. 2019.